

Validated-by-Design

A Shift-Left Approach to Computer System Validation for Cloud-Native GxP Platforms

White-Paper · Paper 2 of 3 — Technical Reference Architecture

Authors: Steven Schmidt, Corrado Iorizzo, Gupta Tulluri

Audience: Architects and Platform Engineers · Enterprise, Solution and Data Architects · Cloud and
Data Platform Engineering · Technical CSV and Quality Engineering stakeholders

Table of Contents

About This Paper 3

1. Architecture Principles for Validated-by-Design 4

2. Policy-as-Code as the Validation Substrate..... 6

3. Data Lineage by Design..... 8

4. Data Consistency by Design 10

5. Reference Architecture..... 12

About This Paper

This is Paper 2 of a three-part series on Shift-Left Computer System Validation, written so each paper can be read on its own.

The shared thesis across the series: Traditional project-centric and document-heavy CSV approaches can struggle to keep pace with the rate of change in cloud-native GxP platforms; the remedy is to engineer validation evidence into the architecture itself, so data lineage and data consistency become inherent properties of the platform rather than artefacts produced by inspection.

The series is organised by audience:

- **Paper 1 — Vision & Operating Model:** for executive and Quality leadership. Why traditional CSV is breaking, what shift-left CSV is, the operating model and governance, the adoption roadmap, and the call to action.
- **Paper 2 — Technical Reference Architecture (this paper):** for architects and platform engineers. Architecture principles, policy-as-code, data lineage and consistency, and the reference architecture.
- **Paper 3 — Regulatory & Implementation Guide:** for CSV, Quality, and Regulatory teams. Regulatory mapping, risks and mitigations, and implementation detail.

This paper builds on the vision and operating model set out in Paper 1 and provides the technical depth for architects and platform engineers. Readers who need the regulatory and implementation detail are pointed to Paper 3.

1. Architecture Principles for Validated-by-Design

Five architectural principles turn the thesis into investable, testable capabilities — each mapped to a regulatory expectation, a concrete implementation pattern, and the anti-pattern it prevents.

Note: all five principles align with EU Annex 11, 21 CFR Part 11, and GAMP 5; the specific clauses each principle supports are noted with each principle below.

Regulatory currency. Clause references in this paper follow the current EU Annex 11 (2011). A significant revision of Annex 11, a revised Chapter 4, and a new Annex 22 on Artificial Intelligence entered public consultation on 7 July 2025 (comment period closed 7 October 2025), so this numbering will change when the revision is finalised. The direction of travel reinforces the thesis here: the revised Annex 11 elevates Security, Identity and Access Management, Qualification and Validation, Audit Trails, and Supplier and Service Management — the same themes the five principles above are built on.

1. Trusted Foundation (Pre-qualified baseline) — Business outcome: faster project onboarding.

GxP applications can leverage and reuse qualification evidence from a continuously qualified landing-zone baseline, rather than requalifying common infrastructure controls from scratch for every workload. In practice this means defining a governed landing zone with approved identity, network segmentation, encryption, logging, monitoring, backup, resiliency, and change-control patterns, then continuously testing and evidencing that baseline so every workload consumes pre-approved platform capabilities. It supports EU Annex 11 expectations for validation, risk management, access control, auditability, and supplier/service management, and GAMP 5 lifecycle assurance by establishing a controlled, risk-assessed baseline. The anti-pattern it prevents is every application team designing and validating its own infrastructure controls independently.

2. Controlled Change (Declarative and immutable infrastructure) — Business outcome: safe, auditable change.

Every change is a versioned, reviewable artefact, with no in-place mutation. Environments are built with infrastructure-as-code, pull-request review, automated testing, signed build artefacts, and controlled approvals, and are rebuilt from source-controlled templates rather than patched by hand — making configuration reproducible and inspection-ready. It supports Annex 11 expectations for change control and audit trails, 21 CFR Part 11 expectations for trustworthy electronic records, and GAMP 5 expectations for controlled configuration and traceability. The anti-pattern it prevents is administrators making emergency console changes that are later rationalised in documentation.

3. Compliance by Design (Policy-as-Code controls) — Business outcome: consistent compliance.

Where regulatory or procedural controls are technically machine-evaluable, they can be translated into policies enforced at deploy and runtime. Regulatory clauses and SOP controls become policy definitions with clear ownership, versioning, approval, enforcement mode, exemption process, and evidence output; the policies prevent non-compliant deployments, detect drift, remediate standard configurations, and escalate exceptions. It supports Annex 11 requirements for security, access control, audit trails, and change control, and GAMP 5 critical thinking by linking each technical control to an intended use and risk-based control objective. The anti-pattern it prevents is policies that exist only as advisory checks — no owner, no alerting, and no expiry for exemptions.

4. Continuous Assurance (Evidence-on-emit) — Business outcome: continuous inspection readiness.

Control evaluations, deployments, data movements, and access events emit signed evidence to an append-only store. Evidence is captured automatically from platform services and operational workflows wherever validated activities occur — for example CI/CD pipelines, policy engines, identity systems, data platforms, operational telemetry, and change approvals — and stored in a time-stamped, access-controlled, append-only repository with retention, integrity checks, and queryable inspection views. It supports Annex 11 expectations for audit trails and data integrity, 21 CFR Part 11 expectations for secure, computer-generated records, and GAMP 5 expectations for objective evidence across the lifecycle. The anti-pattern it prevents is evidence assembled manually at release time from screenshots, exports, and disconnected ticket comments.

5. Built-in Traceability (Lineage-by-substrate) — Business outcome: faster inspections.

Data lineage is captured by the data plane itself, not derived after the fact. Catalogues, change data capture, schema registries, event logs, metadata standards, and governed transformation pipelines emit provenance as data moves, so QA attests to the lineage-producing substrate rather than reconstructing lineage manually during inspection. It supports Annex 11 expectations for audit trails and data integrity and aligns with ALCOA+ — attributable, legible, contemporaneous, original, accurate, complete, consistent, enduring, and available data. The anti-pattern it prevents is lineage reconstructed after an audit request by interviewing teams and reconciling partial logs.

Together they shift validation evidence from a project-by-project activity toward a platform capability: the platform continuously enforces approved controls and generates trusted evidence and built-in traceability as intrinsic characteristics of its design, reducing reliance on project-specific documentation and manual evidence generation.

2. Policy-as-Code as the Validation Substrate

Policy-as-code is one of the mechanisms that makes shift-left real. Where a regulatory or procedural control is technically machine-evaluable, it can be translated into an executable control that prevents, detects, or remediates non-compliance and emits evidence automatically.

Policy-as-Code is the mechanism that turns shift-left CSV from an aspiration into an operating model. Where a control is technically machine-evaluable, the core translation chain is: regulatory requirement → control objective → policy definition → enforcement mode → evidence record. A clause establishes the obligation; the control objective states what must be true in the platform; the policy definition makes that objective machine-evaluable; the enforcement mode determines whether the platform audits, blocks, or remediates non-compliance; and the evidence record proves what was evaluated, when, against which version of the policy, and with what result. Controls that depend on scientific judgement, procedural execution, or contextual Quality decisions remain subject to appropriate human oversight and are not replaced by policy-as-code.

A worked example: Annex 11 §12.1

Take EU Annex 11 §12.1 as a worked example. The clause requires that physical and/or logical controls restrict system access to authorised persons. The control objective can be stated as: “Only approved identities, using approved authentication and role assignments, may deploy or administer GxP platform resources.” In an Azure implementation, that objective becomes a policy initiative composed of several policy definitions: deny public administrative endpoints, deny resources without managed identity where required, require diagnostic settings for access events, restrict role assignments to approved groups, and require privileged access mechanisms for administrative roles. The initiative is assigned at the GxP landing-zone scope, with exemptions requiring CSV approval and expiry dates. When a deployment violates the initiative, Azure Policy can deny the request before the resource is created, and the policy evaluation, assignment version, target resource, requester, timestamp, and compliance state are emitted to a governed evidence store.

Choosing the enforcement mode

The enforcement mode matters. Preventive controls use deny for conditions that should never enter a regulated environment, such as unencrypted storage, unrestricted public access, or deployment outside approved regions. Detective controls use audit where the risk is better managed through visibility, trending, and review, such as identifying non-standard tags, emerging configuration drift, or lower-risk deviations during early adoption. Remediation controls use `deployIfNotExists` when the safe pattern is to add a required supporting control automatically, such as diagnostic settings, backup configuration, monitoring agents, or private endpoint dependencies. For GxP, deny is appropriate where non-compliance would invalidate the control baseline; audit is appropriate where human review is needed to interpret context; `deployIfNotExists` is appropriate where automated remediation is deterministic, approved, and itself evidenced.

Governing policies as QMS artefacts

Policies must be governed as controlled QMS artefacts, not merely engineering scripts. The accountable process or system owner defines the control intent and risk rationale with appropriate Quality oversight. Platform Engineering owns the technical implementation, testing, deployment, and monitoring of the policy. Quality, Security, Data Governance, CSV and Application Owners provide review and approval according to their respective responsibilities and where their domains

are affected. Approval should occur through the same controlled change process used for regulated configuration baselines: versioned policy source, peer review, impact assessment, test evidence, approval record, release note, effective date, and rollback path. Change history must show who proposed the policy, who reviewed it, what requirement it maps to, which environments it applies to, which effect is active, and whether existing workloads require remediation.

Anti-patterns to avoid

The main anti-patterns are predictable. Policy sprawl occurs when teams create overlapping controls without a catalogue, owner, or rationalised control objective. Exemptions without expiry become permanent bypasses and silently erode the validated baseline. Policies that only audit without alerting produce evidence that nobody acts on. Overly aggressive deny policies can block legitimate delivery and push teams toward shadow platforms. The remedy is a policy catalogue with clear ownership, risk classification, enforcement mode, test coverage, exemption rules, and operational review cadence.

Figure 5.1 — Policy-as-Code translation chain



A clause becomes a machine-evaluable control that emits evidence of what was checked, when, and with what result.

3. Data Lineage by Design

Lineage should be an emergent property of the data substrate — captured as data moves — not reconstructed after the fact during inspection.

In a GxP system, data lineage should not be treated as a downstream cataloguing exercise performed after the system is built. It should be a property of the data substrate itself. In regulatory terms, lineage supports ALCOA¹+ expectations by making data attributable, original, accurate, complete, consistent, enduring, and available. It shows where data came from, who or what created it, how it was transformed, which controls applied, and whether derived data can be traced back to its source. In engineering terms, lineage is the provenance graph across systems: the machine-readable path from source event to intermediate transformation to final analytical output, including schema, version, timestamp, actor, processing logic, and dependency metadata.

Three substrates that produce lineage

- The first substrate choice is event sourcing with an immutable log. Instead of storing only the latest state, the system records each state-changing event as an append-only fact. This supports GAMP 5 critical thinking because the control focus moves to whether the event model, retention, access controls, and replay logic are fit for intended use and proportionate to patient or product risk. It also supports EU Annex 11 §9 expectations for audit trails by preserving the sequence of relevant changes and making reconstruction possible from the primary operational record, not from a manually assembled audit package. An immutable event log is not, by itself, a compliant GxP audit trail; the audit-trail design must also ensure appropriate user or system attribution, timestamps, reason-for-change where required, reviewability, retention, protection from alteration, and validation for the intended use.
- The second substrate choice is change data capture with a schema registry. CDC captures changes from source systems and publishes them into controlled downstream pipelines, while the schema registry records the structure, compatibility rules, and version history of the data contract. This pattern makes lineage observable at the point of movement: what changed, in which source, under which schema, and which downstream consumers received it. For GAMP 5, the critical question becomes whether the CDC mechanism, schema governance, and consumer contracts are validated for the intended risk. For Annex 11 §9, the value is that data movement and transformation events are recorded as part of the operating substrate.

¹ **ALCOA+** is a widely used data integrity framework in regulated industries, especially life sciences and GxP environments. It defines the qualities that regulated data should have so it can be trusted during operations, quality review, and inspection.

The original **ALCOA** principles are:

- **Attributable** — you can identify who or what generated, changed, or approved the data.
- **Legible** — the data can be read and understood throughout its lifecycle.
- **Contemporaneous** — the data is recorded at the time the activity happens, not recreated later.
- **Original** — the first capture or a verified true copy is preserved.
- **Accurate** — the data correctly reflects the activity or observation.

The “+” extends those principles with:

- **Complete** — all required data, metadata, changes, and context are retained.
- **Consistent** — data follows expected sequence, format, timestamps, and logic.
- **Enduring** — data remains preserved and usable for the required retention period.
- **Available** — data can be retrieved for review, audit, or inspection when needed.

- The third substrate choice is a **unified catalogue wired into ingest and transformation pipelines**, such as Microsoft Purview or an equivalent vendor-neutral data-governance platform. The catalogue should not be populated only by periodic scans after the fact. It should receive lineage metadata from ingestion jobs, transformation engines, orchestration tools, schema registries, data-quality services, and access-control systems. This creates a continuously updated map of data assets, dependencies, transformations, owners, classifications, and usage. The catalogue becomes inspection-ready because lineage is captured as data flows, not reconstructed when an auditor asks for it.

How QA's role changes

This changes QA's role. In the traditional model, QA often helps reconstruct lineage during inspection by collecting screenshots, exports, interface specifications, transformation notes, and SME explanations. In the shift-left model, QA attests to the substrate that produces lineage. The assurance question becomes: are the lineage-producing mechanisms validated, governed, monitored, and protected from unauthorised change? The accountable process or system owner defines the control intent with appropriate Quality oversight, and Quality determines whether the resulting evidence is acceptable; QA no longer needs to rebuild the chain manually for each inspection scenario.

Worked example: a lab result, end to end

Consider a QC laboratory result flowing from an instrument to a LIMS, then into a data lake and finally into an analytics layer. The instrument generates the original QC result with device identifier, analytical method, timestamp, operator or system account, sample or batch context, and run metadata. The LIMS records receipt, review status, correction history, and sample association. CDC publishes the approved result to the data lake under a registered schema, preserving source identifiers and version metadata. Transformation pipelines enrich or aggregate the result while emitting job IDs, code versions, input-output mappings, and quality checks into the catalogue. When analytics consumes the curated dataset, the report can trace each value back through the graph without bespoke instrumentation. Lineage is no longer an after-the-fact explanation; it is an operating property of the GxP data platform.

4. Data Consistency by Design

Consistency should be engineered into the integration architecture — bounded, observable, and evidenced — rather than proven later through periodic reconciliation.

Data consistency by design means that consistency is engineered into the integration architecture rather than proven later through periodic reconciliation. In GxP terms, this supports EU Annex 11 §5 expectations for data processing, Annex 11 §7 expectations for data storage, and the ALCOA+ principles of **Consistent** and **Complete**. Regulated data must follow an expected sequence, retain required context, remain protected through transfer and storage, and be available with sufficient metadata to explain what happened. The assurance question is therefore not only “did the reconciliation report pass?” but “does the architecture prevent, detect, or bound inconsistency at the moment data is created, moved, transformed, and consumed?”

Patterns that produce consistency

Several architectural patterns make this practical.

The transactional outbox pattern ensures that a business write and the event representing that write are committed atomically to the same primary store. A publisher then emits the event to downstream consumers without risking the classic failure mode where the database commit succeeds but the integration message is lost.

Idempotent consumers allow downstream systems to process the same event more than once without corrupting state, because each event carries a stable identifier and processing logic recognises duplicates.

Exactly-once semantics, where supported by the platform, can reduce duplicate processing further, but should not be assumed universally.

Schema registries with backward-compatibility checks prevent breaking changes from entering regulated data flows.

Contract testing in CI verifies that producers and consumers still agree on message structure, mandatory fields, meaning, and allowed evolution before code reaches a GxP environment.

When eventual consistency is acceptable

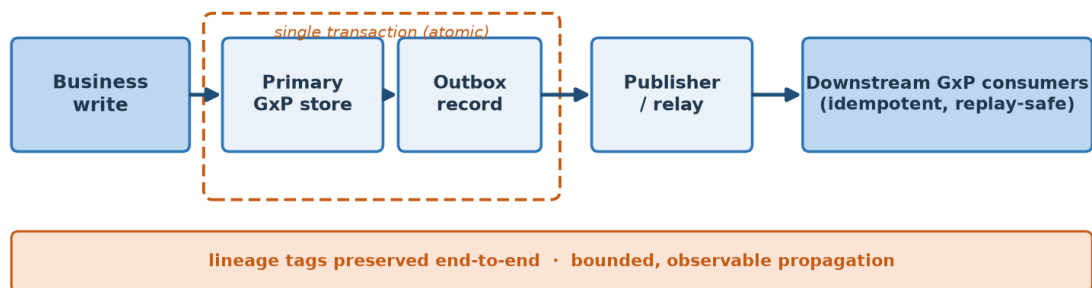
Eventual consistency can be acceptable in GxP when it is both **bounded** and **observable**. Bounded means the maximum acceptable propagation delay, retry window, and failure-handling path are defined, justified by risk, approved, and monitored. For example, a laboratory result may need near-real-time propagation to a review workflow, while a non-critical analytics copy may tolerate a defined delay. Observable means the platform can show, in audit terms, when the source write occurred, when the event was emitted, which consumers received it, whether processing succeeded, whether retries occurred, and whether any message exceeded the approved consistency window. Without those boundaries and signals, “eventual consistency” becomes an excuse for uncertainty. With them, it becomes a controlled operating characteristic.

Handling the residual cases

Residual cases still require compensating actions and human-in-the-loop decision-making. Some failures cannot be safely remediated automatically: conflicting updates, ambiguous mappings, late-arriving critical data, schema violations, duplicate clinical observations, or messages that affect batch

disposition or patient-relevant decisions. In these cases, the platform should quarantine the event, preserve the original payload, raise an alert, assign an accountable owner, record the decision, and emit the outcome as evidence. Human intervention is not a weakness in the model; it is the controlled path for cases where automated consistency would be unsafe or scientifically inappropriate.

Figure 7.1 — Consistent-by-Design integration flow



A business write and its event commit atomically; downstream consumers process idempotently within an approved consistency window.

Checklist for a CSV lead:

- Is the system of record clearly defined for each regulated data element?
- Are writes and outbound events made atomic through an outbox or equivalent pattern?
- Are consumers idempotent, with duplicate detection and replay behaviour tested?
- Are schemas versioned, governed, and checked for backward compatibility before deployment?
- Are producer-consumer contracts tested in CI and linked to release evidence?
- Is any eventual consistency window risk-assessed, approved, monitored, and reportable?
- Are lineage tags preserved across each hop and visible in the catalogue or evidence store?
- Are exceptions quarantined with accountable human review and documented disposition?

5. Reference Architecture

A validated-by-design GxP platform is best read as four cooperating logical layers, each owning distinct controls and emitting its own evidence, so assurance is produced continuously from the bottom up — with intended use, GxP impact and risk assessment determining how much of that reusable evidence a given workload can leverage.

A shift-left, validated-by-design GxP platform is best read as four cooperating logical layers, from a continuously qualified foundation up to the GxP applications. Each layer owns a distinct set of controls and emits its own evidence, so assurance is produced continuously from the bottom up rather than assembled at the end. **The reference architecture described in this section is deliberately vendor-neutral.** A worked example of how these layers can be realised on a specific technology stack is given separately at the end of the section, so the architectural thinking remains portable across platforms, suppliers and industries.

Positioning the core message. The platform does not remove the need for application-level validation. It establishes reusable qualified controls and continuously generated evidence that applications can leverage as a common baseline. Validation effort then shifts to what is specific to the intended use, the associated GxP impact and risk, and the application-level controls, rather than repeating the same qualification activities for every workload.

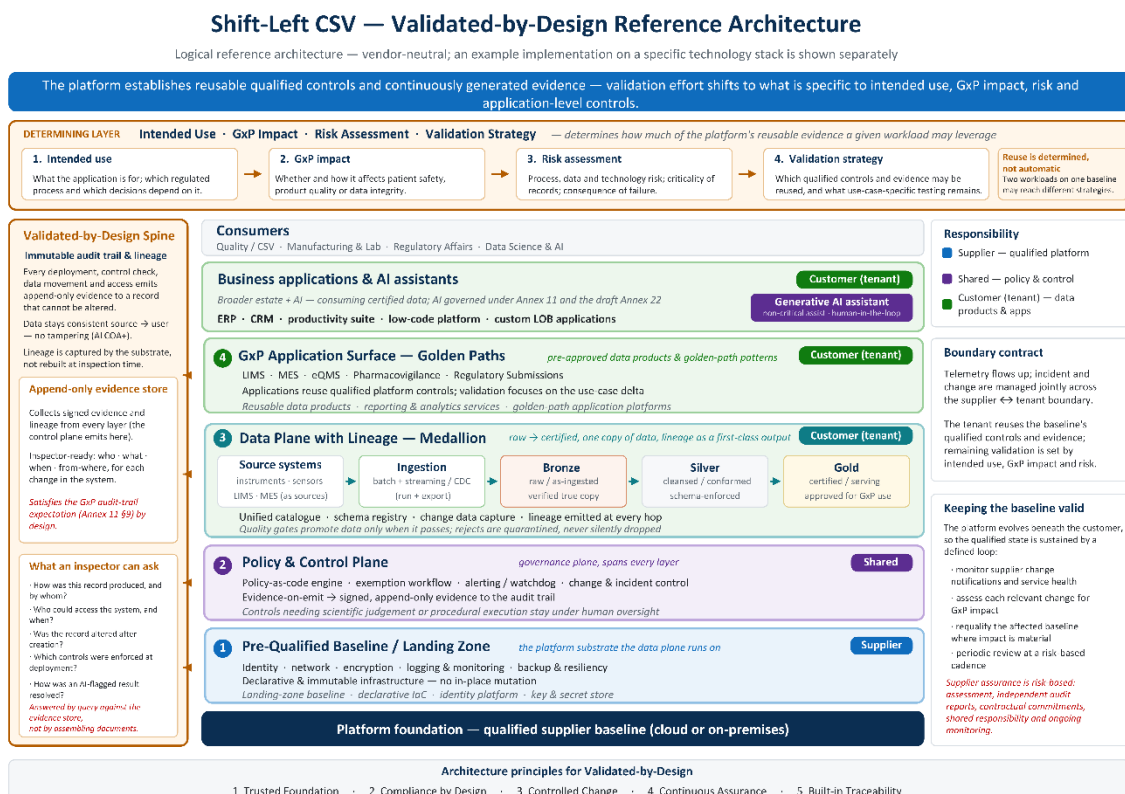


Figure 5.1 — Logical reference architecture (vendor-neutral): the validated-by-design GxP stack

1. Pre-Qualified Baseline / Landing Zone

The foundation is a continuously qualified landing-zone baseline whose approved controls and qualification evidence can be reused by GxP workloads, reducing the need to repeatedly qualify common infrastructure capabilities for each project. It establishes the baseline controls as governed, version-controlled configuration: centralised identity with role-based access and privileged-access management (centralised directory with just-in-time privileged access); segmented, policy-governed networking with private connectivity (segmented virtual networks with private endpoints); encryption of data at rest and in transit with managed keys (a managed key and secret store); baseline logging and monitoring wired to a central workspace (a central log and telemetry workspace); and declarative, re-deployable baseline definitions (templated, declarative infrastructure definitions). Because the baseline is built from templates and re-tested on every change, each workload consumes pre-approved capabilities, and the qualification is evidenced automatically whenever the landing zone is deployed or updated.

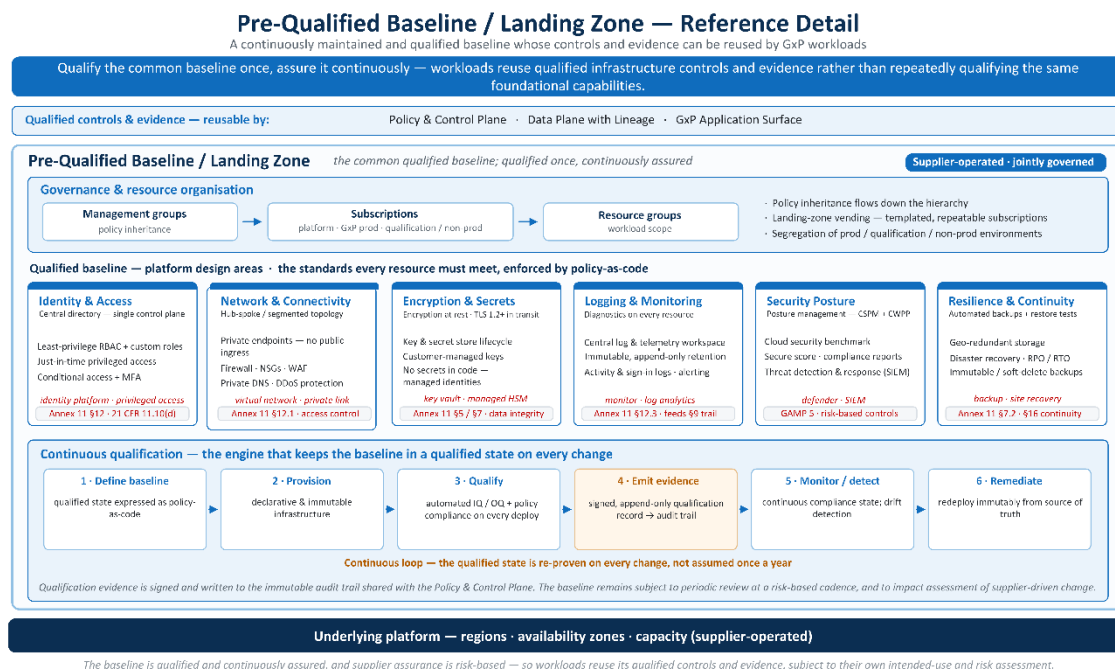


Figure 5.2 — Layer 1: Pre-Qualified Landing Zone

2. Policy & Control Plane

Above the landing zone, the control plane turns regulatory intent into enforced, evidenced controls. A policy-as-code engine evaluates changes at deploy and runtime in audit, prevent, and remediate modes, and every evaluation is emitted as evidence. An exemption workflow carries owner, risk rationale, and expiry (a controlled exemption register) so justified deviations do not silently erode the baseline; drift detection and compliance-state reporting surface divergence (compliance-state reporting and posture management); and alerting and escalation on control failure (alerting and workflow automation) ensure failures are acted on rather than merely recorded. Evidence is written continuously to a tamper-evident store.

Critically, the validator is itself validated. The policy engine and the evidence store are qualified, version-controlled GxP systems in their own right — change-managed and attested to by QA — so the control plane that enforces validation is held to the same standard it enforces. This closes the “who checks the checker” gap that automated compliance would otherwise open.

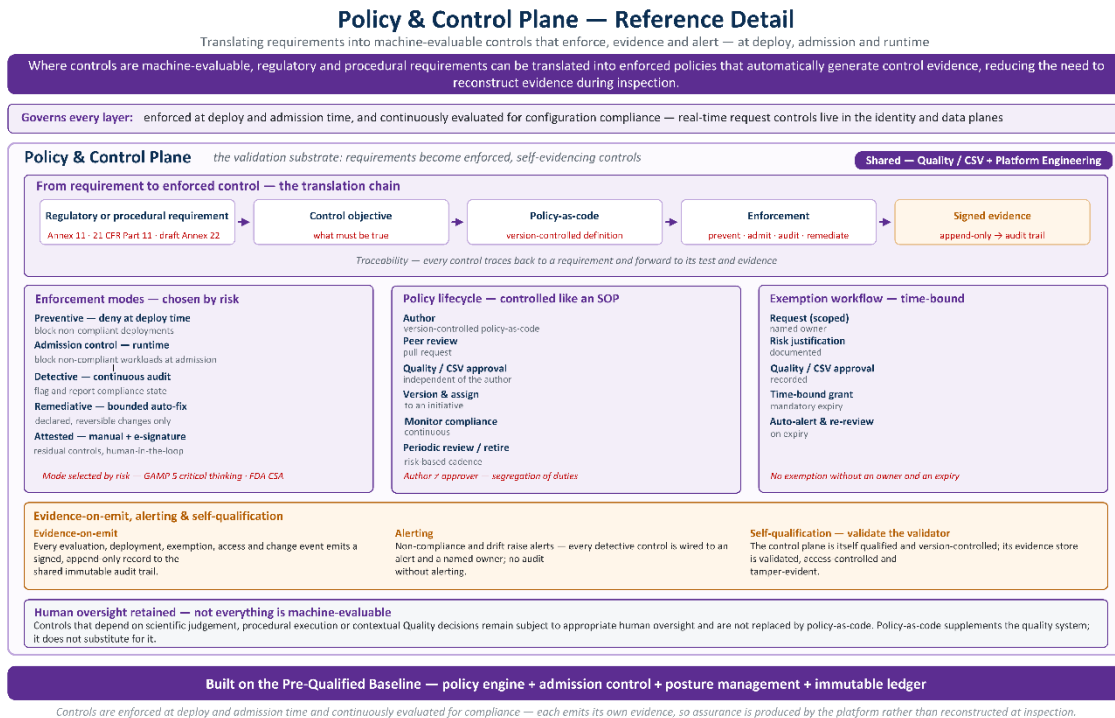


Figure 5.3 — Layer 2: Policy & Control Plane

3. Data Plane with Lineage

The data plane makes lineage and provenance a first-class output of normal data movement rather than a reconstruction exercise. Data flows through a medallion progression — source → ingestion → Bronze (a source-aligned raw copy — completeness and fidelity checked on ingest through checksums, row counts, and schema validation, with the original preserved) → Silver (conformed) → Gold (governed, approved data products) → serving — and at each hop the pipelines emit metadata (source, schema, version, actor, timestamp, and transformation logic) into a governed catalogue (a governed enterprise catalogue). A schema registry enforces contracts and backward compatibility (a schema registry on the streaming platform); change data capture and an event backbone preserve the ordered history of changes; an immutable event log retains that ordered history (Event Hubs Capture to append-only storage); and data-quality and classification rules emit evidence as data moves. Data integrity is engineered in — ALCOA+, Delta transaction-log change history, controlled reprocessing, and risk-based retention and archival — and QA attests to this substrate instead of rebuilding the chain for each inspection.

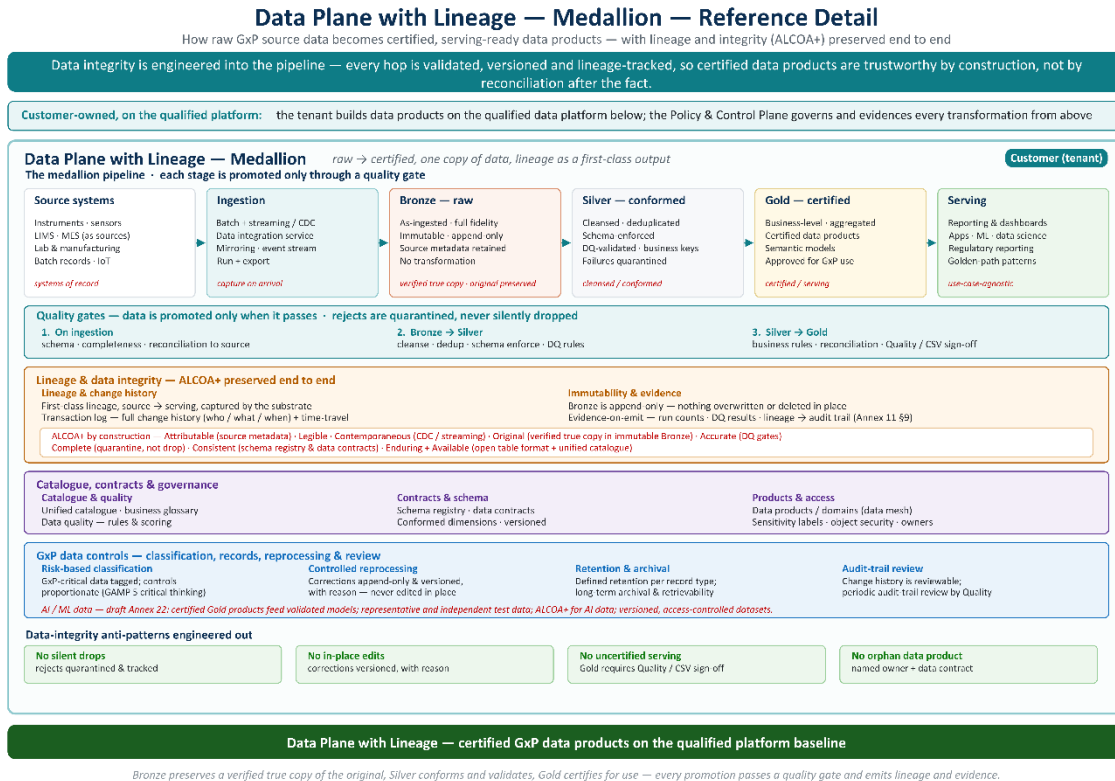


Figure 5.4 — Layer 3: Data Plane with Lineage

4. GxP Application Surface

At the top, delivery teams build GxP workloads on pre-approved golden paths rather than assembling controls themselves. A golden path packages the compliant pattern — reusable landing-zone controls, policy bindings, evidence emission, and lineage wiring — into a reusable template for common workloads such as LIMS, MES, eQMS, pharmacovigilance, regulatory submissions, and analytics. Validation inheritance reduces duplication by allowing application teams to leverage qualified infrastructure, approved platform services, reusable golden-path components and existing supplier evidence. GAMP categorisation should be applied at the appropriate component level, together with intended use, complexity and risk, to determine the proportionate qualification and validation activities required for the workload. The surface provides self-service provisioning within guardrails, pre-wired application logging and audit-trail patterns, controlled workflow and e-signature building blocks — which must meet 21 CFR Part 11 signature-manifestation and signature-to-record-linking requirements to serve as compliant electronic signatures (see Paper 3) — and reusable integration patterns such as transactional outbox and idempotent consumers, so the compliant path is the fastest path.

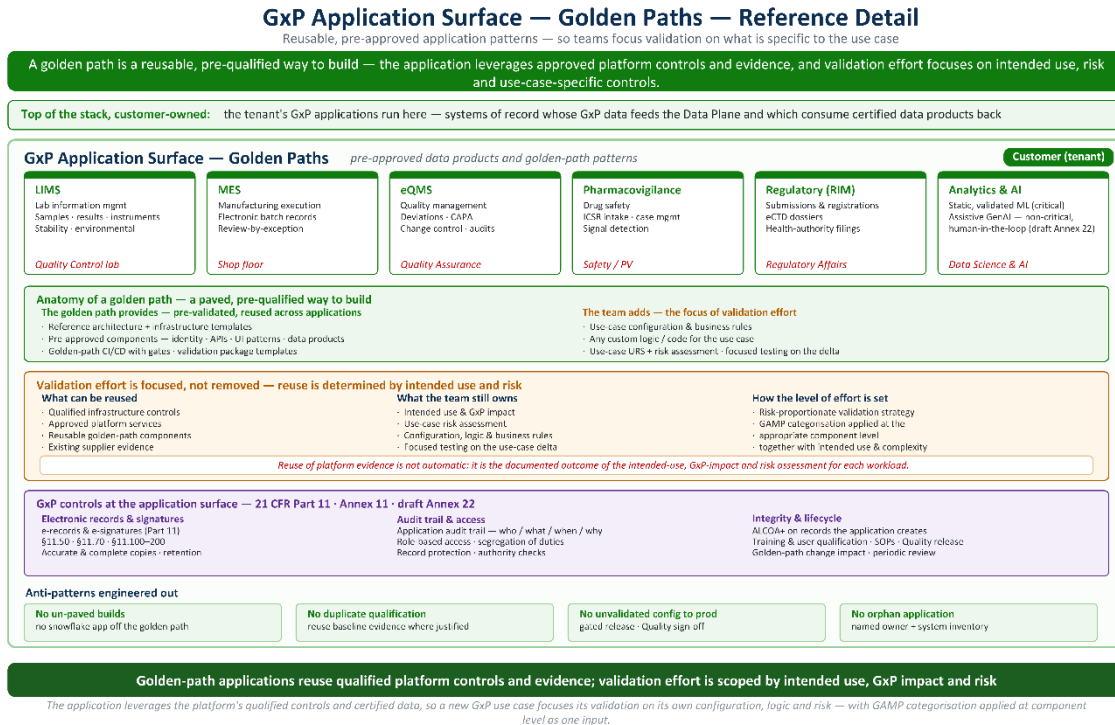


Figure 5.5 — Layer 4: GxP Application Surface

5. Intended Use, GxP Impact and Validation Strategy

Alongside and above the four technical layers sits the determining layer of the architecture: the assessment that decides how much of the platform's reusable evidence a given workload may actually leverage. The chain is intended use → GxP impact → risk assessment → validation strategy.

Intended use defines what the application is for, which regulated process it supports, and which decisions depend on it. **GxP impact** establishes whether and how the workload affects patient safety, product quality or data integrity. **Risk assessment** evaluates the associated process, data and technology risk, including the criticality of the records and the consequences of failure. **Validation strategy** then determines, on a risk-proportionate basis, which inherited controls and platform evidence can be reused as a baseline, and which additional use-case-specific qualification, testing and documentation are required.

This layer is the mechanism that makes validation inheritance defensible. Reuse of platform evidence is not automatic and is not a property of the architecture alone: it is an outcome of a documented assessment for each workload. Two applications running on the same qualified baseline may legitimately arrive at very different validation strategies because their intended use, GxP impact and risk profiles differ. GAMP categorisation is applied at the appropriate component level as one input into that assessment, together with intended use, complexity and risk, rather than being inferred from the platform layer a component happens to sit on.

The practical consequence for delivery teams is that the platform answers the question “**what can I reuse, and what evidence already exists?**” while this layer answers the question “**what must I still do for this particular use case?**” Keeping the two questions distinct is what prevents the architecture from being read as a claim that application-level validation is no longer required.

AI workloads and the draft Annex 22

The same requirement → control → enforcement → evidence chain governs AI. The draft EU GMP Annex 22 (published for consultation in July 2025 as additional guidance to Annex 11; the consultation period closed in October 2025 and the text is not yet final) proposes that AI/ML used in GMP-critical applications should be limited to static, deterministic, validated models, and indicates that dynamic or continuously-learning and probabilistic models would be excluded from critical use. The draft further states that a critical model should perform at least as well as the validated manual process it replaces, with acceptance criteria (for example sensitivity, specificity and accuracy) defined for the intended use and SME-approved before testing. For generative AI and copilots, the draft indicates these would not be appropriate for critical GMP decisions, and describes their use as non-critical assistance with a qualified human-in-the-loop, grounded on certified, governed data. Because the annex remains in draft, organisations should track the final text and treat the current expectations as direction of travel rather than settled requirement. The underlying controls — defined intended use, representative and independent test data, ALCOA+ for AI data, explainability, continuous performance monitoring, change control, and human oversight — can be enforced and evidenced through the platform like any other control.

Example implementation: Microsoft Azure and Microsoft Fabric

Beneath the four layers sits the platform foundation. In this example implementation, the underlying Azure platform provides globally certified infrastructure with data-residency options (regions and availability zones, the EU Data Boundary, and independent third-party attestations such as Germany's BSI C5), physical and compute isolation, durable encrypted storage, a resilient network backbone, and a continuously-compliant control plane — supplier evidence that can support the qualified-baseline argument. Microsoft Fabric provides a managed SaaS data platform with security, governance, lifecycle and compliance capabilities that can form part of the qualified platform baseline: OneLake as one open Delta copy (with shortcuts, mirroring, and Direct Lake), governed by Purview, OneLake security, Microsoft Entra, and customer-managed keys, with change control across the SaaS boundary handled through safe-deployment rings and tenant application lifecycle management. The regulated user remains responsible for assessing the service, defining its intended GxP use and establishing the required qualification and validation evidence.

Discharging supplier assessment for a hyperscaler. For hyperscale cloud providers, supplier assurance should follow a documented risk-based approach. Depending on criticality and practical access, this may combine direct supplier assessment, independent third-party audit reports and certifications (for example ISO/IEC 27001, SOC 1 and SOC 2, and BSI C5), contractual commitments, a documented shared-responsibility model recording which controls the supplier operates and which the customer retains, and ongoing service monitoring rather than a one-time assessment. The regulated user remains accountable for determining whether this evidence is sufficient for the intended GxP use.

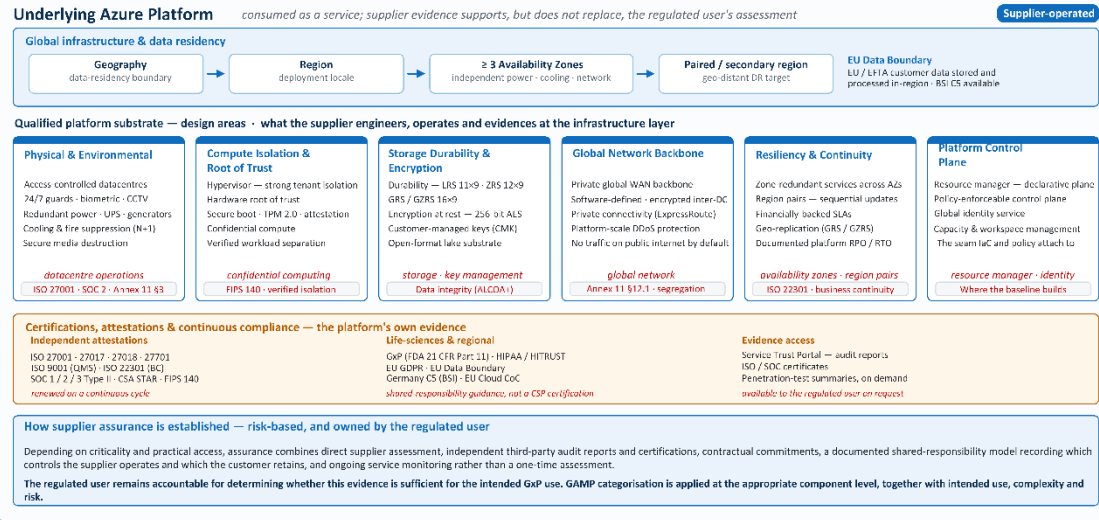
Keeping the qualified baseline valid over time. Because the platform evolves beneath the customer through vendor-driven change, the qualified state of the baseline is sustained by a defined loop: monitor vendor change notifications and service-health channels, assess each relevant change for GxP impact, and requalify the affected baseline where the impact is material — with a periodic review at a risk-based cadence confirming that the baseline, policies, and evidence pipeline remain in their validated state. This closes the gap that safe-deployment rings and tenant lifecycle management open when services change without customer action.

Example Implementation — Underlying Cloud Platform (Microsoft Azure)

The supplier-operated infrastructure on which the qualified baseline is built — one concrete realisation of the logical foundation

Supplier assurance for hyperscale infrastructure is risk-based — combining supplier assessment, independent audit reports and certifications, contractual commitments, a documented shared-responsibility model and ongoing monitoring

Below the tenant boundary: the supplier owns and operates the physical facilities, hardware root of trust, hypervisor, global network and control planes; the tenant does not configure this layer



Supplier-operated global cloud — datacentres · 24/7 site-reliability and security operations

Supplier evidence and attestations support the qualification of the baseline built on this infrastructure; they do not by themselves establish the qualified state of the regulated user's systems.

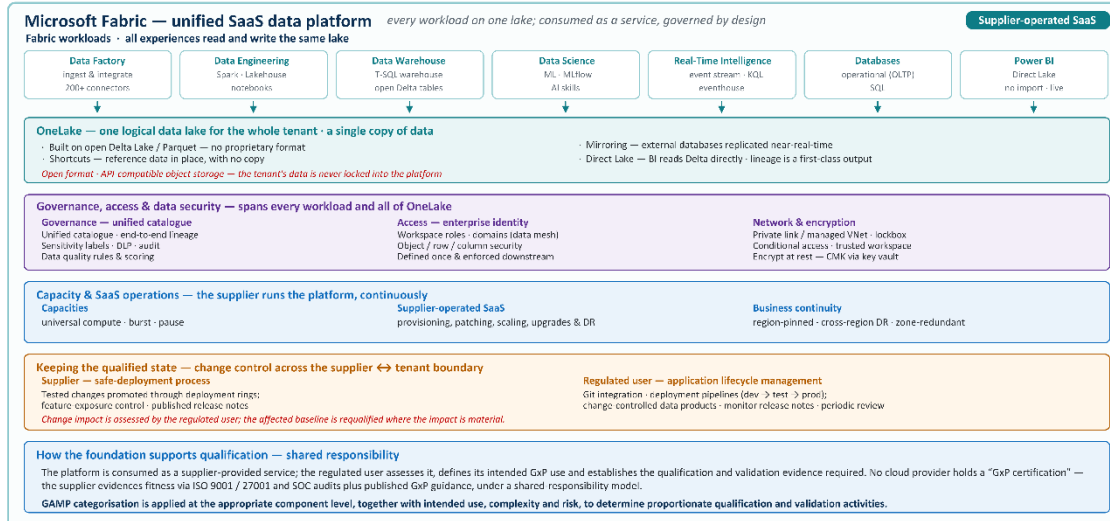
Figure 5.6 — Example implementation: platform foundation on Microsoft Azure

Example Implementation — Managed SaaS Data Platform (Microsoft Fabric)

A managed SaaS data platform that can form part of the qualified platform baseline — one concrete realisation of the logical data plane

Microsoft Fabric provides a managed SaaS data platform with security, governance, lifecycle and compliance capabilities that can form part of the qualified platform baseline.

Part of the supplier-operated foundation: the regulated user assesses the service, defines its intended GxP use, and establishes the required qualification and validation evidence



A managed platform baseline — the regulated user determines its qualified use

The platform is kept current by the supplier through a safe-deployment process; the regulated user assesses change impact, change-controls its own data products, and maintains the qualified state of the baseline it relies on.

Figure 5.7 — Example implementation: Microsoft Fabric as part of the platform baseline

Reading the architecture

For an executive audience, the architecture reduces to four stacked bands — Pre-Qualified Landing Zone, Policy & Control Plane, Data Plane with Lineage, and GxP Application Surface — with evidence emitted continuously up through every band into a single evidence store and inspection view, so validation scales with delivery rather than trailing it. For an engineering audience, each band decomposes into concrete components: identity, network, encryption, logging, and monitoring in the landing zone; the policy engine, exemption workflow, evidence pipeline, and alerting in the control plane; ingest → change data capture → schema registry → event backbone → catalogue in the data plane; and golden-path workloads consuming inherited controls at the surface — with evidence connectors running from every component to the shared, append-only evidence store. Cutting across all four layers is an immutable audit-trail and lineage spine, and a clear responsibility split: Microsoft owns the platform, policy and control are shared, and the customer owns its data and applications.